Decision trees and random forests

A focused guide to the formulas used in applications A, B, and C

1. A tree is a sequence of questions

A tree routes an input through tests such as "queue length $q \le 3$?" and "payload $s \le 5$ MB?". Each terminal node, or *leaf*, stores a prediction learned from the training rows that reach it. Depth counts tests on the longest path; a root-only tree has depth zero. A stump has one split and two leaves.

A tree can represent the conditional mean in application A:

$$f(s,q) = \begin{cases} 40 \text{ ms}, & q \le 3, \\ 60 \text{ ms}, & q > 3,\ s \le 5, \\ 90 \text{ ms}, & q > 3,\ s > 5. \end{cases}$$

Payload matters only when the queue is long: the tree expresses an interaction without requiring a product feature. Ordinary numerical tests create axis-aligned regions and constant predictions within each region. They do not smoothly extrapolate beyond the observed feature range.

Regression: predict the mean inside a leaf. For rows A in a leaf, minimizing $\sum_{i\in A}(y_i - c)^2$ gives

$$\boxed{c_A = \bar{y}_A, \qquad \mathrm{RSS}(A) = \sum_{i\in A}(y_i - \bar{y}_A)^2.}$$

Indeed the derivative with respect to c is $2|A|c - 2\sum_{i\in A} y_i$. For a feature j and threshold t, split into $A_L = \{i : x_{ij} \le t\}$ and $A_R = \{i : x_{ij} > t\}$. Choose the admissible split with smallest $\mathrm{RSS}(A_L) + \mathrm{RSS}(A_R)$. Reject empty or too-small children.

A split you can verify. Take $x = (1,2,3,4,5,6)$ and $y = (2,3,2,8,9,10)$. Parent RSS is $208/3$.

Threshold	1.5	2.5	3.5	4.5	5.5
Child RSS total	53.2	39.25	8/3	25.25	46.8

The winning split is $x \le 3.5$. The leaf predictions are $7/3$ and 9; training RMSE is $\sqrt{(8/3)/6} = 2/3$. The RSS reduction also equals

$$\boxed{\frac{n_L n_R}{n_L + n_R}(\bar{y}_L - \bar{y}_R)^2,}$$

so a split helps when the child means differ, with their sizes taken into account.

Efficient search. Sort each feature; test midpoints only between distinct values. Prefix sums $S_k = \sum_{i\le k} y_{(i)}$ and $Q_k = \sum_{i\le k} y_{(i)}^2$ give left RSS $Q_k - S_k^2/k$ and the analogous right RSS. Center responses first to reduce cancellation from a large common offset. Sorting costs $O(n \log n)$; scanning then costs $O(n)$ per feature.

Remember. Split selection fits the training data. A low training RSS is not an estimate of how well the tree predicts new observations.

2. Classification: keep the leaf probabilities

For a binary target $y \in \{0, 1\}$, a leaf estimates the positive probability as

$$\boxed{\widehat{\pi}_A = \frac{\#\{i \in A : y_i = 1\}}{|A|}.}$$

Classify as 1 if $\widehat{\pi}_A \geq t$ under our explicit tie convention. The threshold t is a decision choice; it is not part of learning the split criterion. A leaf with one positive row gives probability 1 but very little evidence about future certainty. Minimum leaf size helps control this variability.

Gini impurity measures class mixing. Write $G(A)$ for the Gini impurity of node A. For class fractions $p_1, \ldots, p_K$,

$$G(A) = 1 - \sum_{k=1}^{K} p_k^2, \qquad G(A) = 2\widehat{\pi}_A(1 - \widehat{\pi}_A) \quad \text{in the binary case.}$$

Gini is zero for a pure leaf and largest when classes are balanced. It is the probability that two independent draws from the leaf's empirical label distribution disagree. Entropy $H(A) = -\sum_k p_k \log p_k$, with $0 \log 0 = 0$, is another common measure; it also favors purer groups.

Weight child impurities by their sizes. Here $I(A)$ is a *generic name for the chosen impurity*, not a new measure: use $I(A) = G(A)$ for Gini, or $I(A) = H(A)$ for entropy. With $n_A = |A|$ and child sizes n_L, n_R, maximize

$$\boxed{\text{gain} = I(A) - \frac{n_L}{n_A} I(A_L) - \frac{n_R}{n_A} I(A_R).}$$

Without these weights, a tiny pure child would receive too much influence. For regression, $I(A) = \text{RSS}(A)/n_A$ gives the same split ranking as minimizing total child RSS.

A better probability split can leave accuracy unchanged.

Node	Positive	Negative	$\widehat{\pi}$	Gini
Parent	8	2	0.8	0.32
Left child	5	0	1	0
Right child	3	2	0.6	0.48

Weighted child Gini is $0.5(0) + 0.5(0.48) = 0.24$, so gain is 0.08. At threshold 0.5 both children still predict 1, leaving the same two mistakes. Gini detects a useful change in probability structure that immediate accuracy misses.

Categorical inputs need explicit treatment. In Carseats, shelf location is nominal. One-hot indicators such as `ShelveLoc_Good` permit the test "Good versus not Good". Arbitrary integer codes silently impose a split ordering. Fit the encoder on the fitting rows and specify what happens to a category not seen there. The notebooks use all-zero indicators for an unknown category.

The target is $Y = \mathbf{1}\{\texttt{Sales} > 8\}$, so `Sales` must be excluded from the inputs. A predictor measured only after the outcome would also be unusable for a genuine advance prediction.

Remember. Use leaf probabilities for probability metrics and cost-sensitive decisions. A hard class prediction throws away their degree of certainty.

3. Control a tree's complexity

Tree construction is greedy: choose the best current split and repeat in each child. The first split is not reconsidered after later splits. A useful deeper structure may require an initially weak split, so greedy fitting need not find the best possible tree. Small changes in training rows can also change early tests.

Two direct controls. Smaller maximum depth limits the number of tests; larger minimum leaf size requires each prediction to average more observations. More complex trees usually fit training data better but may fit noise. Strictly increasing feature transformations preserve candidate partitions in exact arithmetic, so trees usually need no standardization. One-hot encoding and feature availability still matter.

Pruning puts a price on every leaf. Grow a larger tree and consider its subtrees. Minimize

$$\boxed{R_\alpha(T) = R(T) + \alpha L(T), \qquad \alpha \geq 0,}$$

where $L(T)$ is the leaf count. In regression, $R(T) = \sum_\ell \mathrm{RSS}(A_\ell)/n$; for Gini classification, $R(T) = \sum_\ell (n_\ell/n) G(A_\ell)$. Larger α favors fewer leaves. The regression penalty has squared-response units; the Gini penalty has a different scale. Using raw RSS instead of normalized RSS multiplies equivalent penalties by n.

For the six-row example, consider a tree with a root test at 3.5 and an additional right-branch test at 4.5:

Subtree	Leaves	$R(T)$	Penalized risk
Keep both tests	3	$7/36$	$7/36 + 3\alpha$
Keep root only	2	$4/9$	$4/9 + 2\alpha$
No tests	1	$104/9$	$104/9 + \alpha$

The first two options tie at $\alpha = 1/4$; the last two at $100/9$. Choose the smaller tree at a tie. These are training tradeoffs among subtrees; cross-validation is still needed to select predictive complexity.

The same evaluation discipline as regression applies.

1. Make 60/20/20 train/validation/test partitions before target exploration; stratify binary labels in these independent-data examples.
2. Compare a prespecified parameter grid using five-fold training CV. Grow a fresh tree and fit preprocessing inside each fold.
3. Select by RMSE for regression or Brier score for binary probabilities. Fit the selected configuration on the training partition.
4. Use validation for an audit, probability/threshold checks, or exploratory importance. Freeze the model and policy before the final test.

Do not grow a full-data tree first and reuse its learned splits across CV folds. Repeated customers or future-time prediction need appropriate group/time splits.

Remember. Pruning changes the tree. Cross-validation chooses the setting. A separate test estimates the performance of the resulting frozen procedure.

4. Bagging and random forests: average unstable trees

Bagging. Draw n training indices *with replacement*, fit a tree, and repeat B times. Repeated indices retain their multiplicity. Average the predictions:

$$\widehat{f}_B(x) = \frac{1}{B}\sum_{b=1}^{B} \widehat{f}_b(x), \qquad \widehat{\pi}_B(x) = \frac{1}{B}\sum_{b=1}^{B} \widehat{\pi}_b(x).$$

Regression averages numerical responses; classification here averages positive probabilities. Always check class-column ordering when reading probability arrays.

Averaging probabilities is not the same as majority voting. For five tree probabilities $(0.2, 0.3, 0.4, 0.9, 0.95)$, the average is 0.55, so the averaged-probability rule predicts 1 at threshold 0.5. Thresholding each tree first gives three votes for 0 and two for 1: majority voting predicts 0. Record which rule the implementation uses.

Why averaging helps. At one fixed input, suppose each fitted tree prediction has variance σ_T^2 across repeated training experiments, and pairs have common correlation ρ. Then

$$\boxed{\operatorname{Var}\left(\frac{1}{B}\sum_{b} \widehat{f}_b\right) = \sigma_T^2\left(\rho + \frac{1-\rho}{B}\right).}$$

Expand the variance of a sum: B individual variance terms plus $B(B-1)$ covariance terms, all divided by B^2. With variance 16 and correlation 0.25, ensemble variance is 16, 5.2, and 4.12 for $B = 1, 10, 100$; the limit is 4. Adding trees reduces the unshared variation but cannot remove the shared part.

Random forests also randomize candidate features. At every node, offer a random subset of m columns to the split search rather than always offering all p. This can make trees less similar. It can also weaken individual splits, so choose m by predictive evaluation. With $m = p$, bootstrap forests behave as bagged trees under the same controls. The subset is sampled anew at each node, not once for the whole forest.

Two controls with different jobs. Increasing B stabilizes the average and increases time/storage. Changing depth, minimum leaf size, or m changes the fitted rule and its bias/variance tradeoff. Many trees cannot rescue unavailable features, leakage, a wrong target, or irreducible outcome noise.

Do not confuse two notions of correlation. Correlating two trees' prediction vectors across many inputs measures how similarly their rules vary across those inputs. It is not the fixed-input, repeated-training correlation ρ in the formula above. Application C reports that descriptive similarity and separately resamples training data to study prediction variability at fixed inputs.

Remember. Randomization makes diversity possible; averaging makes diversity useful. Whether it helps this prediction problem is an empirical question.

5. OOB evaluation, importance, and decisions

A bootstrap leaves some training rows out. The probability that one row is absent from n draws is

$$q_n = (1 - 1/n)^n \approx e^{-1} \approx 0.368.$$

Its expected OOB tree count is Bq_n. For row i, average only over the trees that did not use it:

$$\widehat{\pi}_i^{\mathrm{OOB}} = \frac{1}{|\mathcal{B}_i|} \sum_{b \in \mathcal{B}_i} \widehat{\pi}_b(x_i), \qquad \mathcal{B}_i = \{b : i \text{ absent from bootstrap } b\}.$$

If $|\mathcal{B}_i| = 0$, its OOB prediction is undefined. Exclude that row from the current OOB metric and report coverage; never replace it by zero. With few trees, the coverage and probability estimates can be unstable. OOB is an internal assessment, not an independent final test after extensive tuning. Learned full-data preprocessing can also compromise its interpretation.

Feature importance answers a specific question. Mean decrease in impurity (MDI) totals training split gains credited to a feature, weighted by the fraction of training mass at each node, then normalizes across features. It can favor features with many candidate thresholds and reflect overfit.

For validation permutation importance under Brier loss, use

$$\mathrm{PI}_j = \mathrm{Brier}(y_{\mathrm{val}}, \widehat{\pi}(X_{\mathrm{val}}^{\mathrm{perm(j)}})) - \mathrm{Brier}(y_{\mathrm{val}}, \widehat{\pi}(X_{\mathrm{val}})).$$

Permute one column, predict without refitting, and repeat the permutation to measure Monte Carlo variability. Positive importance means the shuffle hurt. Correlated predictors can substitute for each other; shuffling one also creates unusual combinations. Jointly shuffle a redundant pair with one shared row permutation to assess reliance on the pair while preserving their relationship. Neither method proves causation; MDI and permutation loss have different scales.

Evaluate probability quality and decision quality separately.

$$\mathrm{Brier} = \frac{1}{m} \sum_i (y_i - \widehat{\pi}_i)^2, \qquad \text{cost per row} = \frac{c_{\mathrm{FP}}\mathrm{FP} + c_{\mathrm{FN}}\mathrm{FN}}{m}.$$

Pure leaves can give exactly 0 or 1. A wrong probability-zero event has infinite theoretical log loss. The notebooks explicitly clip to $[10^{-12}, 1 - 10^{-12}]$ for reported log loss, applying the same rule to every candidate.

With appropriate probabilities and zero cost for correct decisions, the cost-minimizing threshold is $c_{\mathrm{FP}}/(c_{\mathrm{FP}} + c_{\mathrm{FN}})$. Carseats uses costs 1 and 3, giving $t = 0.25$. Its notebook also selects a validation threshold, then freezes it. Report confusion counts, precision, recall, and cost rather than accuracy alone.

Remember. Training importance, internal OOB scores, and final test performance are different quantities. State which data and which loss produced each result.

6. The three companion notebooks

Each notebook runs independently and places task statements before readable code. Simulation seed is 17 and split seed is 23. Tables and figures are saved after execution. Full task details and conventions are in the notebooks.

A. A service with regime changes

`02_A_service_regimes.ipynb`

$$s \sim U(0,10), \quad q \sim \text{Poisson}(3), \quad \epsilon \sim N(0,8^2), \qquad y = 40 + 20\mathbf{1}\{q > 3\} + 30\mathbf{1}\{q > 3, s > 5\} + \epsilon.$$

Generate 1,200 independent requests. Verify a prefix-sum stump against every candidate in the six-row example. Compare a mean baseline, a linear model, a stump, and a depth-two tree through their prediction surfaces. Select pruning strength by training CV, then bootstrap training rows to inspect split and prediction stability. Finish with frozen-model test errors and complexity/runtime.

Main lesson: the tree family matches this piecewise-constant simulator. That explains its advantage here; it does not establish a universal ranking.

B. Carseats: categorical features and probabilities

`02_B_carseats.ipynb`

Use the local 400-row `Carseats.csv`; define high sales as $Y = \mathbf{1}\{\texttt{Sales} > 8\}$, giving 164 positives. Exclude sales from inputs. Fit one-hot encoding within folds, compare a logistic baseline, select depth/leaf controls and a pruning penalty using training CV Brier score, and draw a shallow tree with leaf counts and positive fractions.

Use validation to audit probabilities and select an alarm threshold for costs 1 and 3. Compare the frozen policy with prespecified probability baselines on the final test. Translate two paths back into ordinary feature statements.

Main lesson: a tree predicts probabilities from local counts; small leaves and a small evaluation sample demand careful interpretation.

C. Bagging, decorrelation, OOB, and importance

`02_C_forest_diagnostics.ipynb`

Generate 2,000 rows with 20 columns: $x_1, x_3, x_5, \ldots, x_{20}$ are independent $U(-1,1)$ draws, $x_2 = x_1 + N(0, 0.03^2)$, and $x_4 \sim \text{Bernoulli}(0.5)$ is independent. Labels follow

$$Y \mid x \sim \text{Bernoulli}(\pi(x)), \qquad \pi(x) = \sigma\big(-0.7 + 2.5x_1 + 2\mathbf{1}\{x_3 > 0.2\}\big).$$

Compare bagging and forests with 300 trees; select m and minimum leaf size by training CV. Independently build bootstrap trees and audit row-specific OOB averages, coverage, and convergence through 600 trees. Compare tree-rule similarity, resampled prediction variability, MDI, and validation permutation importance, including a joint permutation of (x_1, x_2).

Main lesson: averaging, internal evaluation, and interpretation each require their own checks. Keep generating probabilities out of the input columns.

Further reading. The full second lecture is in `PdfLectures`. The bundled book is James et al., *An Introduction to Statistical Learning with Applications in R*, 2nd ed. (2021), Chapter 8.1, 8.2.1, and 8.2.2; Chapter 5 supplies bootstrap and validation background.